

# HAP 보조 MEC의 온라인 오프로딩: EEDO

Online dynamic multi-user computation offloading and resource allocation for HAP-assisted MEC: an energy efficient approach

---

Si-han Chen and Wanchun Jiang (2024)

EEDO가 에너지와 두 대기열을 함께 다루는 방법

# 원문 범위와 용어

2024년 Journal of Cloud Computing의 공개 version of record를 기준으로 읽는다

| 항목 | 원문에서 확인한 내용                         |
|----|-------------------------------------|
| 문제 | 다중 GD의 부분 오프로딩과 HAP 자원 할당           |
| 방법 | Lyapunov drift-plus-penalty 기반 EEDO |
| 목표 | 장기 기대 에너지 최소화와 queue stability      |
| 평가 | 정적 HAP의 시뮬레이션과 이론 bound             |

---

이 텍스트는 RL이나 실제 비행 실험이 아니라, 온라인 최적화 모델과 그 시뮬레이션을 설명한다

---

# 왜 HAP 보조 MEC인가

원문의 출발점은 고정 지상 MEC가 coverage를 주기 어려운 원격 지역이다

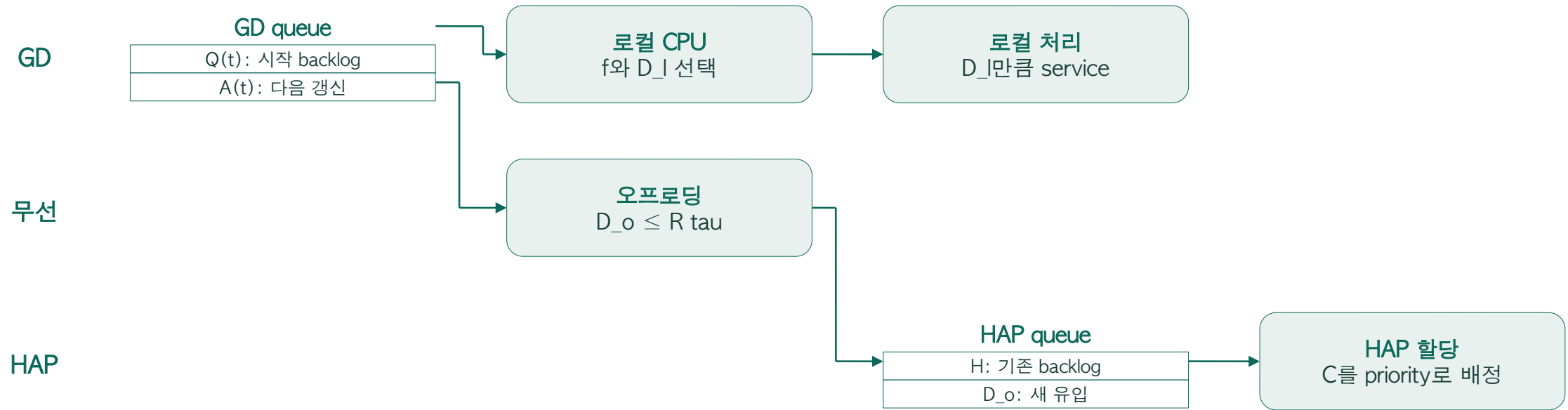
- GD는 battery와 local CPU가 제한된 computing-intensive task를 받는다
- 멀리 있는 cloud로 모두 보내면 전송 지연·에너지·혼잡 문제가 커질 수 있다
- HAP은 넓은 영역을 덮는 airborne edge server로 모델링된다
- 하지만 한 HAP의 CPU와 radio capacity도 유한하다
- 따라서 여러 GD가 보내는 양과 HAP이 처리할 양을 함께 정해야 한다

---

HAP은 넓은 coverage를 주지만, 처리 용량이 제한된 공유 edge다

# 작업의 경로와 두 queue

한 slot에서 GD backlog는 local CPU와 radio로 갈라지고, radio 경로만 HAP-side queue에 더해진다



Q는 GD backlog, H는 HAP backlog다. 둘을 섞으면 제어 규칙을 잘못 읽는다

# 에너지 장부: 세 항을 함께 센다

P1의  $E(t)$ 는 GD local CPU, GD radio, HAP processing의 합이다

| 항     | 읽기 쉬운 식                  | 커지는 이유              |
|-------|--------------------------|---------------------|
| local | $E_l = x_i f^2 \phi D_l$ | CPU 주파수와 local work |
| radio | $E_o = D_o P / R$        | 전송량·전력, 낮은 rate     |
| HAP   | $E_h = l_1 C$            | HAP이 처리한 bit        |
| total | $E = \text{각 GD의 세 항 합}$ | 세 경로를 함께 고려         |

---

오프로딩은 local CPU 에너지를 줄일 수 있지만, radio와 HAP 에너지가 사라지는 것은 아니다

---

# P1: 장기 평균 에너지, 네 feasibility 제약

목적은 long-run expected energy 최소화이며, latency 자체를 직접 최소화하는 식은 아니다

| 제약 | 의미                       | 막는 오류              |
|----|--------------------------|--------------------|
| C1 | $0 \leq f \leq f_{\max}$ | GD CPU 한계 초과       |
| C2 | $0 \leq D_o \leq R \tau$ | 한 slot radio 용량 초과 |
| C3 | $D_o \leq Q$             | 없는 task 전송         |
| C4 | $0 \leq C \leq H$        | 없는 HAP task 처리     |

---

queue stability는 장기 Lyapunov 요구이고, C1–C4는 slot별 feasibility 제약이다

---

# 긴 미래를 매 slot 제어로 바꾸기

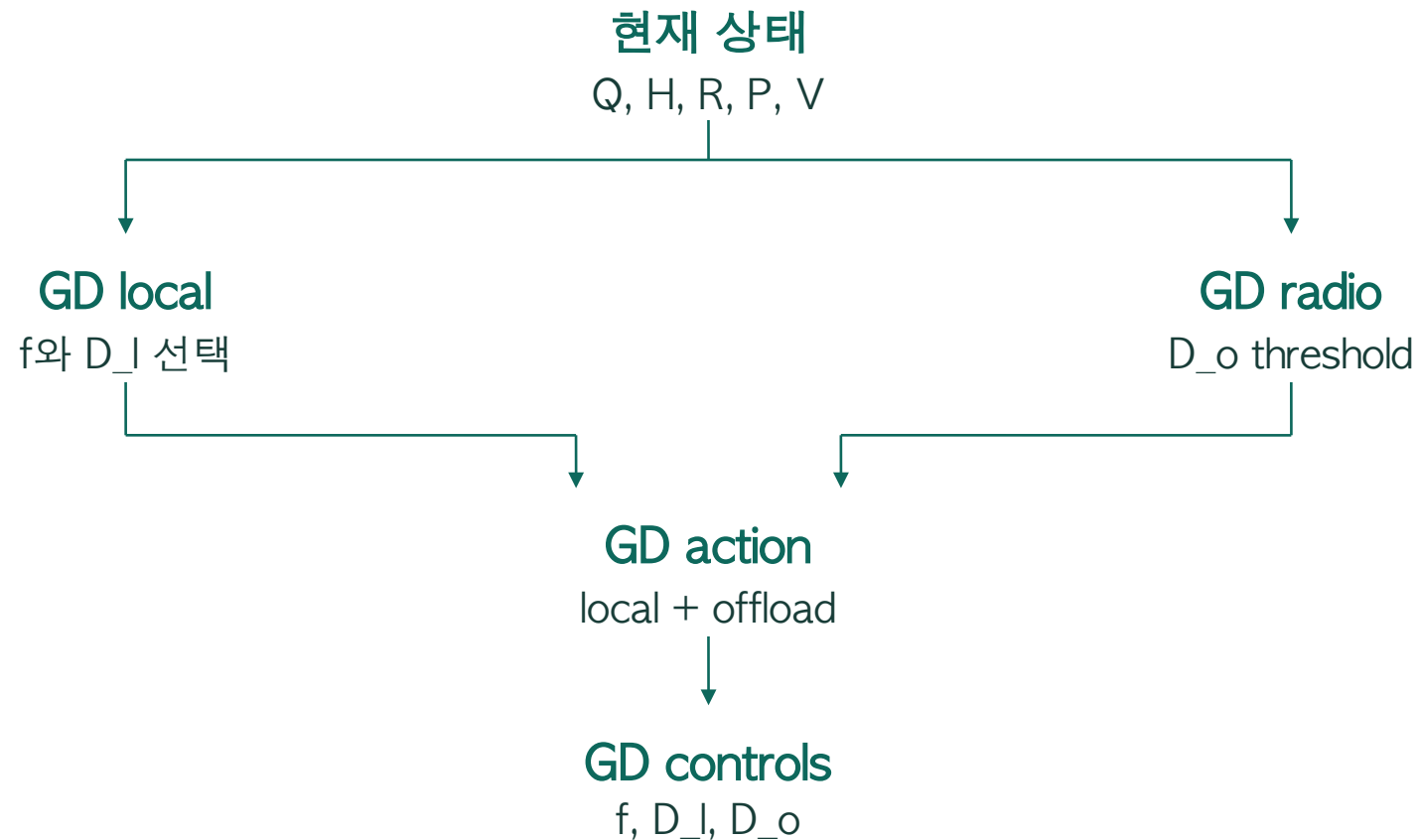
EEDO는 queue 제곱합의 drift에 energy penalty를 더한 상한을 현재 slot에서 낮춘다

- 상태 Theta는 모든 Q와 H backlog를 모은 벡터다
- Lyapunov L은  $Q^2$ 와  $H^2$ 의 합으로 backlog 크기를 수량화한다
- drift는 다음 slot으로 갈 때 L이 얼마나 변하는지의 조건부 기대값이다
- drift-plus-penalty는  $\text{drift} + V \times \text{expected energy}$ 다
- V가 커지면 에너지 항을 더 강하게 중시하는 설계가 된다
- Theorem 1의 upper bound를 최소화해 P2의 세 결정으로 분리한다

미래 task를 예측하지 않고, 현재 backlog와 관측값으로 slot bound를 낮추는 제어다

# EEDO의 GD 결정은 두 갈래다

P2의 GD 부분은 local CPU와 radio offload를 따로 고른다. HAP allocation은 다음 슬라이드에서 다룬다



# 결정 1 : local CPU frequency

Eq. 25는 backlog가 크면 CPU를 올리려 하지만, CPU와 queue가 주는 상한으로 clip한다

- P2의 local 항은  $V \xi f^3 \tau - Q f \tau / \phi$  다
- stationary candidate는 square-root of  $Q$  divided by  $3V\xi \phi$  다
- 최종  $f$ 는 candidate를  $f^*$  이하로 clip한다
- $f^*$ 는 min of  $Q \phi / \tau$  and  $f_{\max}$  다
- local service는  $D_l = f \tau / \phi$  로 바뀐다
- $Q$ 가 큰 쪽은 service를 요구하고,  $V$ 가 큰 쪽은 energy를 더 중시한다

local CPU rule은 backlog와 CPU 에너지의 균형이며, 장치 한계를 넘지 않는다

## 결정 2: offload는 threshold rule

Eq. 27은 한 계수의 부호로 'rate 한계까지 보낼지, 0일지'를 정한다

| 계수              | 0 이하이면                         | 0 초과이면    | 직관                                       |
|-----------------|--------------------------------|-----------|------------------------------------------|
| $V P/R - Q + H$ | $D_o = \min(R \text{ tau}, Q)$ | $D_o = 0$ | radio cost - GD urgency + HAP congestion |
| Q 증가            | offload 쪽으로                    | 해당 없음     | GD backlog가 크면 보낼 유인이 큼                  |
| H 증가            | 해당 없음                          | local 쪽으로 | HAP backlog는 보내기를 억제                     |
| P/R 증가          | 해당 없음                          | local 쪽으로 | 비싸거나 느린 radio는 억제                        |

offload-only가 아닌 이유는 H와 P/R가 같은 방향으로 무조건 전송을 막을 수 있기 때문이다

## 결정 3: HAP의 남은 CPU를 배정하기

Algorithm 1은 유한 capacity  $C_h$  안에서 H backlog가 긴 eligible GD부터 처리한다

|    |          |                                  |
|----|----------|----------------------------------|
| 01 | capacity | $C_h = f_{h,\max} \tau / \phi_h$ |
| ↓  |          |                                  |
| 02 | priority | $V _1 - H$ 오름차순 sort             |
| ↓  |          |                                  |
| 03 | eligible | 계수 0 이하인지 확인                     |
| ↓  |          |                                  |
| 04 | allocate | $C = \min(H, C_h)$               |
| ↓  |          |                                  |
| 05 | repeat   | $C_h$ 를 줄이고 다음 GD                |

HAP은 무한 service 대신, 한 slot의 shared capacity를 priority 순으로 나눈다

# Algorithm 1의 한 slot 읽는 순서

결정 규칙과 queue update를 분리하면 ‘보낸 task가 언제 H에 더해지는지’를 혼동하지 않는다

01    관측                      A, Q, H, R, P를 읽음



02    GD CPU                  Eq. 25로 f와 D\_l



03    GD radio                Eq. 27로 D\_o



04    HAP 배정                C\_h와 Eq. 29



05    queue 갱신              Q는 Eq. 7, H는 Eq. 8

Eq. 8은 H에서 C를 뺀 뒤 D\_o를 더한다. 새 offload를 먼저 service하지 않는다

# 구성 예시: 한 GD의 decision 계산

아래 값은 Eq. 25, 27, 29를 손으로 확인하기 위한 invented teaching trace다

| 입력    | 값                               | 계산                                 | 결정           |
|-------|---------------------------------|------------------------------------|--------------|
| queue | $Q=6, H=4$                      | 기존 backlog                         | 현재 상태        |
| CPU   | $V=x_i=\phi=\tau=1, f_{\max}=1$ | $\sqrt{6/3}=\sqrt{2} \approx 1.41$ | $f=1, D_l=1$ |
| radio | $R_{\tau}=3, V P/R=2$           | $2 - 6 + 4 = 0$                    | $D_o=3$      |
| HAP   | $C_h=2, V l_1=2$                | $2 - 4 = -2$                       | $C=2$        |

각 수는 예시일 뿐이다. 핵심은 square-root clip, offload 부호, HAP priority의 세 규칙이다

# 구성 예시: 두 queue의 다음 값

A=2를 더하고, Eq. 7과 Eq. 8을 각각 적용한다

| queue  | update                | 계산                | 다음 값  |
|--------|-----------------------|-------------------|-------|
| GD Q   | $\max(Q-D_I-D_o,0)+A$ | $\max(6-1-3,0)+2$ | 4     |
| HAP H  | $\max(H-C,0)+D_o$     | $\max(4-2,0)+3$   | 5     |
| 남은 C_h | $C_h-C$               | $2-2$             | 0     |
| 읽는 법   | H의 service 뒤 D_o 유입   | 새 작업도 H에 남음       | 순서 경계 |

---

D\_o=3을 보냈어도 HAP은 C=2만 처리했다. 그래서 H는 4에서 5로 커질 수 있다

# V trade-off: theorem이 실제로 말하는 것

Theorem 2는 arrival rate  $\lambda + \epsilon$ 과 Lemma 1의 조건 아래 energy와 average queue의 upper bound를 준다

- energy:  $E_{\text{EEDO}} \leq E^* + Z/V$
- average queue:  $\bar{J} \leq [Z + V(\hat{E} - E_{\text{check}})] / \epsilon$
- 첫 bound는  $V$ 가 커질수록  $Z/V$  항이 작아진다
- 둘째 bound는  $V$ 를 포함하므로 backlog cost가 커질 수 있다
- $\epsilon$ ,  $\hat{E}$ ,  $E_{\text{check}}$ ,  $Z$ 와 Lemma 1의 존재 조건이 식에 들어간다
- 따라서 ‘ $V$ 를 키우면 항상 더 좋다’는 결론은 theorem의 내용이 아니다

핵심 trade-off는 energy gap의  $1/V$  항과 queue bound의  $V$  항을 함께 읽는 데 있다

# 시뮬레이션 envelope

정적 HAP이 1 km × 1 km remote zone의 무작위 GD를 서비스하는 특정 setting이다

| 범주      | 원문 값                               | 범주     | 원문 값                                     |
|---------|------------------------------------|--------|------------------------------------------|
| HAP     | 20 km, CPU 20 GHz                  | GD CPU | 1 GHz                                    |
| radio   | 0.1 GHz carrier, 100 MHz           | power  | 0.01–0.2 W                               |
| arrival | uniform $[0, 1.8] \times 10^6$ bit | slot   | $\tau = 1$ s                             |
| model   | $\phi = 1000$ cycles/bit           | energy | $\xi = 10^{-27}$ , $l_1 = 10^{-4}$ J/bit |
| channel | LoS/NLoS loss 0.1/21               | noise  | $\sigma^2 = 10^{-13}$ W                  |

결과는 이 channel, CPU, arrival, static-HAP setting에 한정되며 자동 이전되지 않는다

# V를 바꾼 결과: 낮은 energy, 높은 queue

Figs. 2-3은 논문의 simulation에서 V가 커질수록 이 두 지표가 반대 방향으로 움직임을 보인다

| V 증가 | energy           | mean queue       | 원문 해석                    |
|------|------------------|------------------|--------------------------|
| 더 작음 | 상대적으로 큼          | 상대적으로 작음         | queue를 더 강하게 해소          |
| 더 큼  | 감소 추세            | 증가 추세            | energy를 더 우선             |
| 공통   | simulation curve | simulation curve | Theorem 2의 trade-off와 부합 |

Fig. 2-3의 방향 요약일 뿐, 특정 V를 보편적 최적값으로 제시하지 않는다

# 부하와 GD 수가 늘면 무엇이 보이나

Figs. 4–7은 높은 arrival multiplier와 더 많은 GD에서 energy와 mean queue가 커지는 simulation을 보여 준다

| 변화                     | energy         | mean queue     | 원문의 이유                     |
|------------------------|----------------|----------------|----------------------------|
| arrival alpha: 0.6→1.0 | 증가             | 증가             | 더 많은 task 도착               |
| GD 수: 50→120           | 증가             | 증가             | HAP processing capacity 유한 |
| arrival-rate runs      | equilibrium 관찰 | equilibrium 관찰 | Figs. 4–5의 해석              |

---

EEDO는 부하를 없애지 않고, 유한 capacity 아래 부하 변화에 대응하는 제어다

---

# 비교 실험: energy만 보면 틀릴 수 있다

Figs. 8–9는 네 policy의 energy curve와 queue curve를 함께 놓아야 한다

| policy       | 작동                   | energy만 볼 때 | queue와 함께 읽기                  |
|--------------|----------------------|-------------|-------------------------------|
| Local-only   | 새 task를 local 처리     | energy가 더 큼 | EEDO는 이와 비교해 절감               |
| Offload-only | 새 task를 HAP으로 전송     | 낮아 보일 수 있음  | finite HAP capacity로 queue 증가 |
| GTCO-21      | greedy offload       | 낮아 보일 수 있음  | 과도한 offload로 queue 증가         |
| EEDO         | threshold + priority | 절감 목표       | reported run에서 stability 유지   |

낮은 energy curve만 보지 말고, backlog가 계속 쌓이는지도 함께 확인해야 한다

# EEDO가 주장하는 것과 아닌 것

source claim의 범위를 보존하면 online optimization을 과장하지 않고 이해할 수 있다

- 주장: prior statistics 없이 current  $Q, H, R, P$ 로 online decision
- 주장: model의 energy-queue bound 분석
- 주장: static-HAP simulation의 energy-queue 비교
- 비주장: reward를 학습하는 generic RL controller
- 비주장: 모든 실제 HAP network의 latency SLA 보장
- 비주장: HAP CPU-radio capacity가 무한함

current observation을 쓰는 online rule과 prior statistics 불필요는 양립한다

# 마무리: 세 개의 제어와 한 개의 trade-off

EEDO는 partial offloading을 CPU·radio·HAP service 세 결정으로 나누고, V로 energy와 backlog를 조절한다

- 문제: remote-zone GD와 하나의 유한 HAP edge server
- 상태: GD queue Q와 HAP queue H를 분리해 추적
- 제어: local  $f$ , offload  $D_o$ , HAP service  $C$
- 규칙: CPU square-root clip, radio threshold, HAP priority sort
- 근거: conditional Lyapunov bounds와 static simulation figures
- 한계: 모델·channel·parameter·static HAP 가정 밖은 별도 검증이 필요

핵심 질문은 ‘얼마나 보낼까’보다, 어디에 backlog가 있고 어떤 energy를 감수할지다